TEMA 1: MÁQUINAS DE ESTADOS ALGORÍTIMICAS

1.1.- Introducción: Máquinas de estados algorítmicas.

1.2.- Unidad de procesamiento de datos (Ruta de datos).

1.3.- Controladores. Implementación de un controlador.

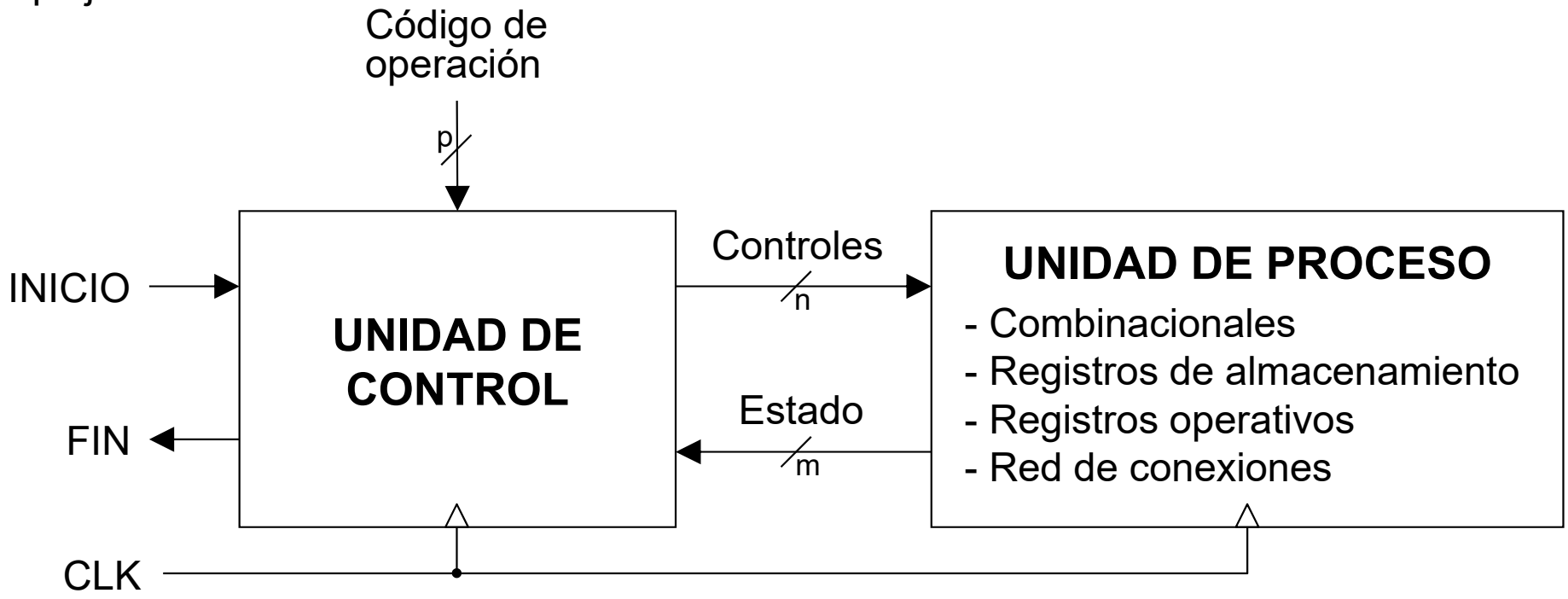
1.4.- Control cableado.

1.5.- Control microprogramado – palabra de control.

UNIDADES DE PROCESO Y CONTROL

A la hora de diseñar sistemas digitales complejos, se divide la tarea a realizar en una serie de pasos simples, cada uno de ellos asociado a un circuito digital sencillo. Todos ellos, con sus correspondientes interconexiones y terminales de control constituyen la ***unidad de proceso***.

La ***unidad de control*** es el sistema secuencial que suministra a la unidad de proceso los niveles lógicos adecuados en los tiempos apropiados para que efectúe la secuencia de operaciones lógicas sencillas (*microoperaciones*) que, en su conjunto, realizan la operación compleja deseada.

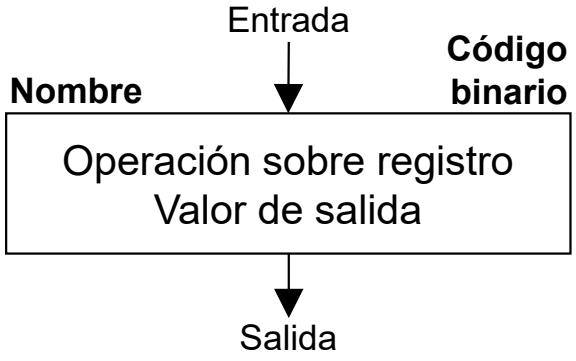


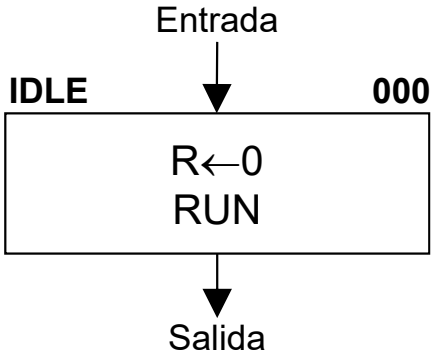
DIAGRAMAS ASM

Normalmente los circuitos digitales complejos se definen mediante *algoritmos*, en los que se especifican una serie de procedimientos y caminos de decisión que, a partir de micooperaciones elementales, permiten realizar una tarea compleja.

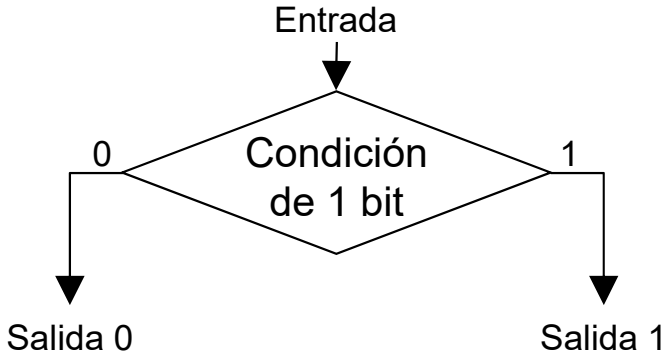
Los algoritmos se representan gráficamente mediante **diagramas ASM** (*Algorithmic State Machines*), también llamados **organigramas**:

Caja de estado:

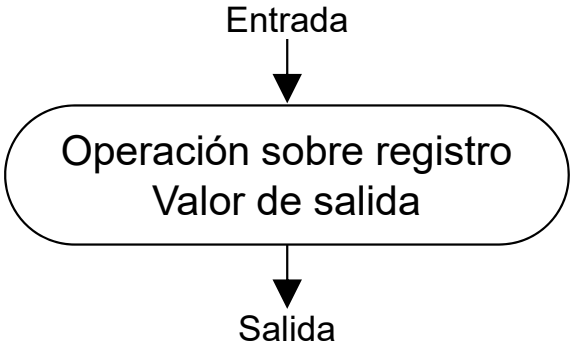




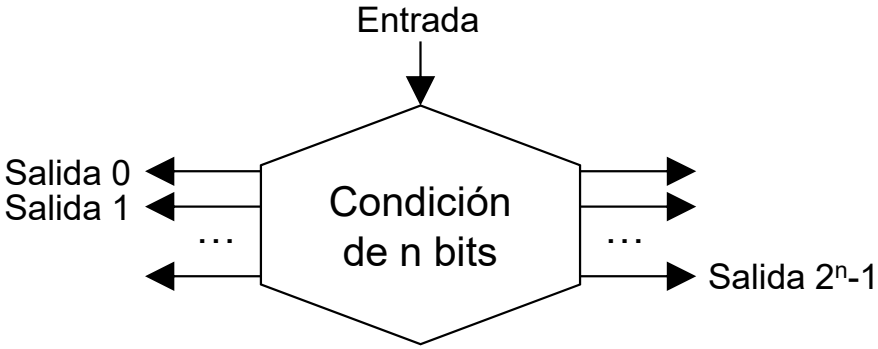
Caja de decisión escalar:



Caja de salida condicional:



Caja de decisión vectorial:



DIAGRAMAS ASM

Un *bloque ASM* consiste en una caja de estado y todas las cajas de decisión y de salida condicional conectadas entre su salida y los caminos de entrada a la misma caja de estado o a otras.

Ejemplo:

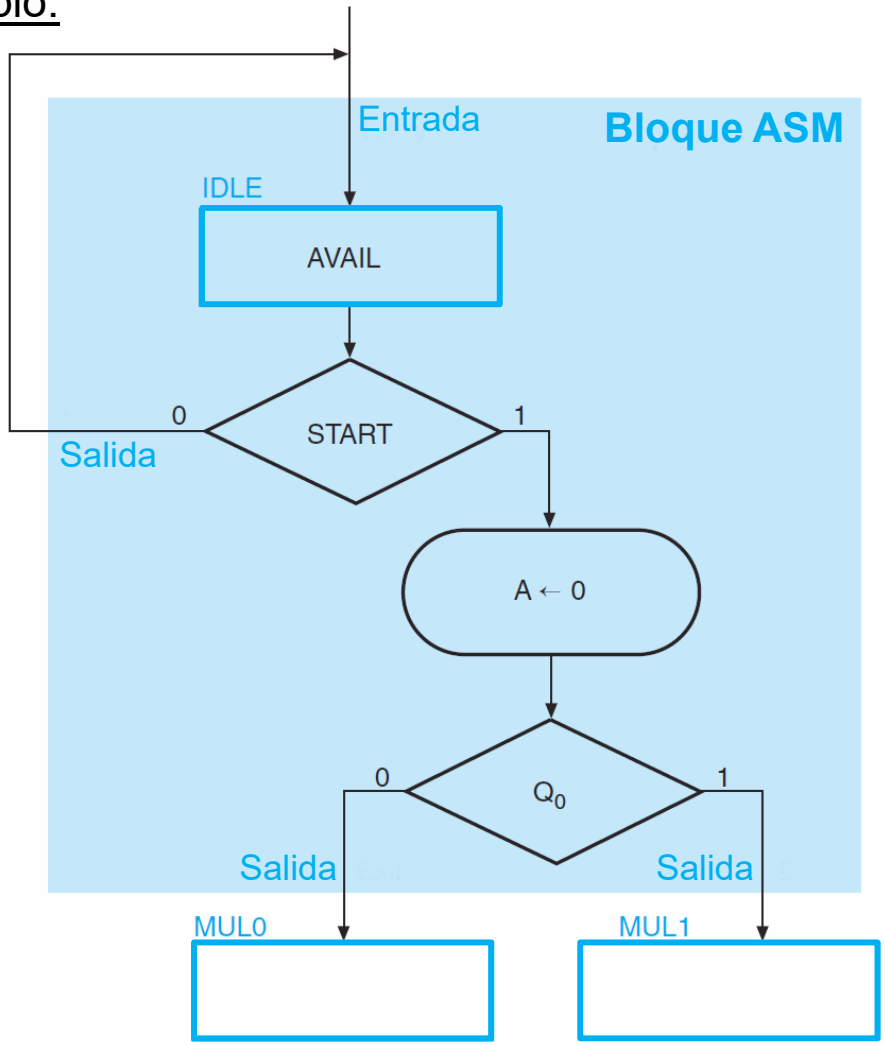
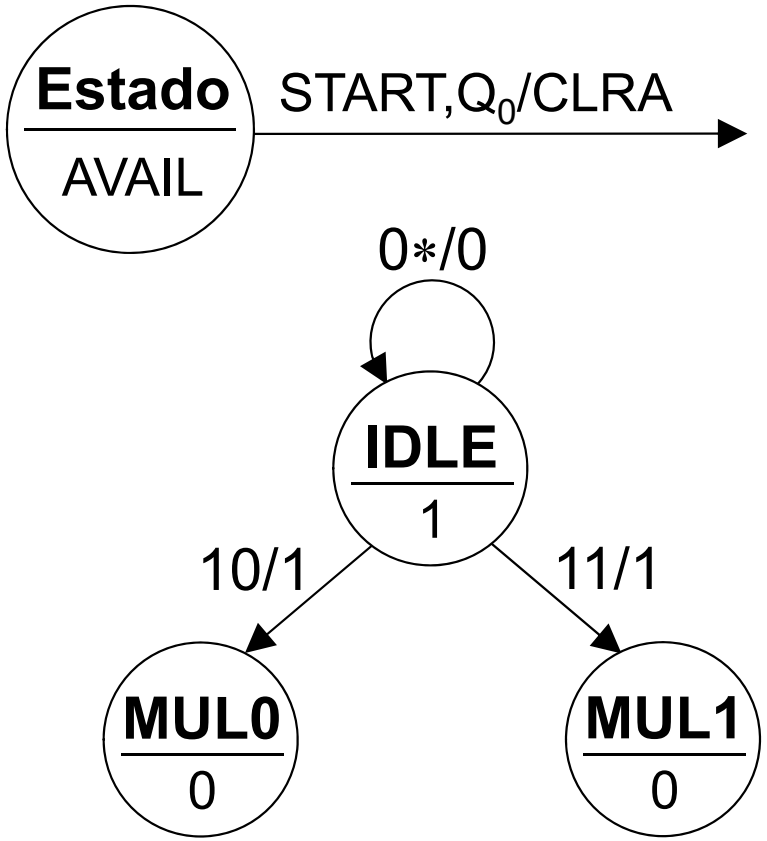


Diagrama de flujo equivalente:

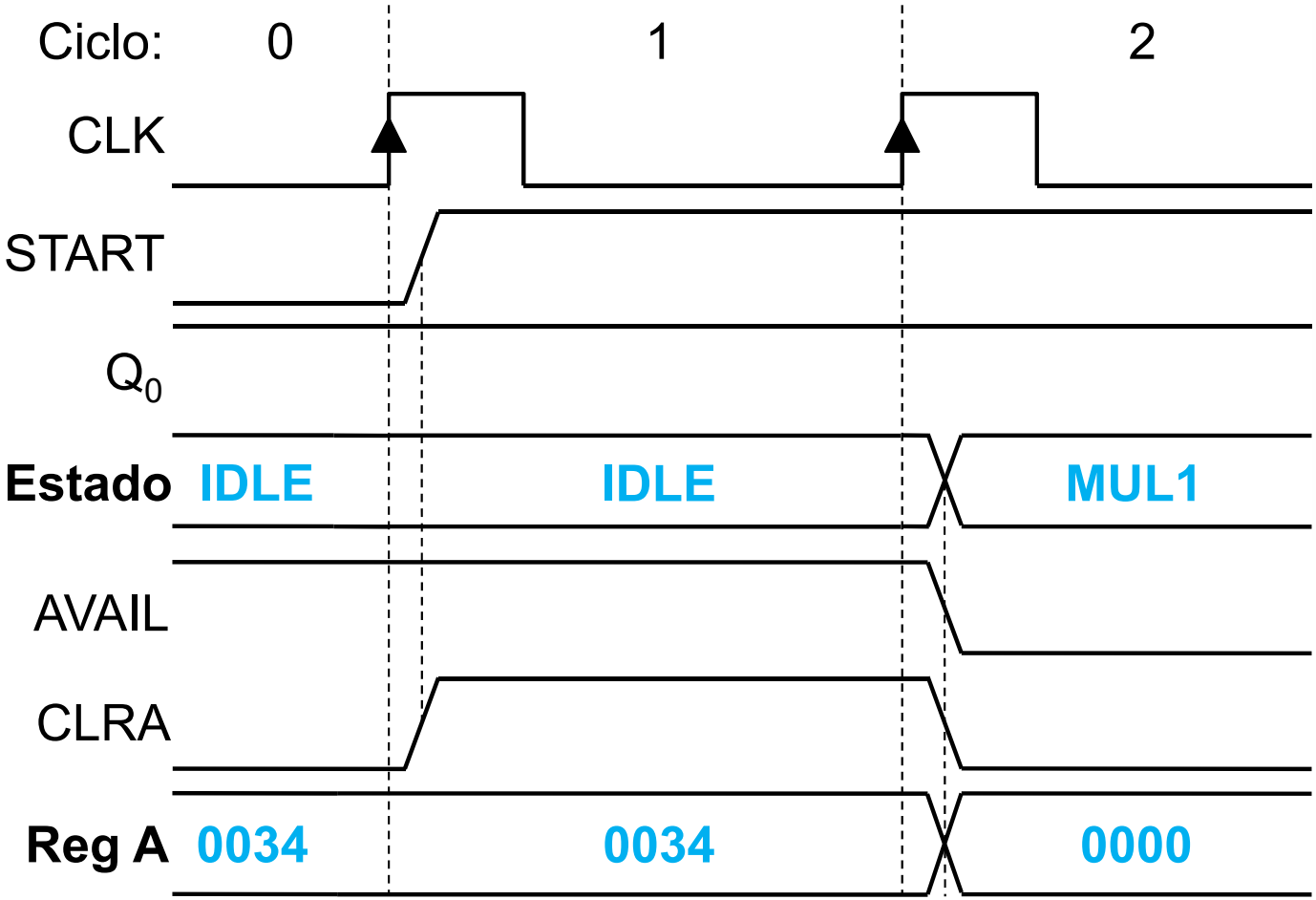
Entradas: START y Q_0.

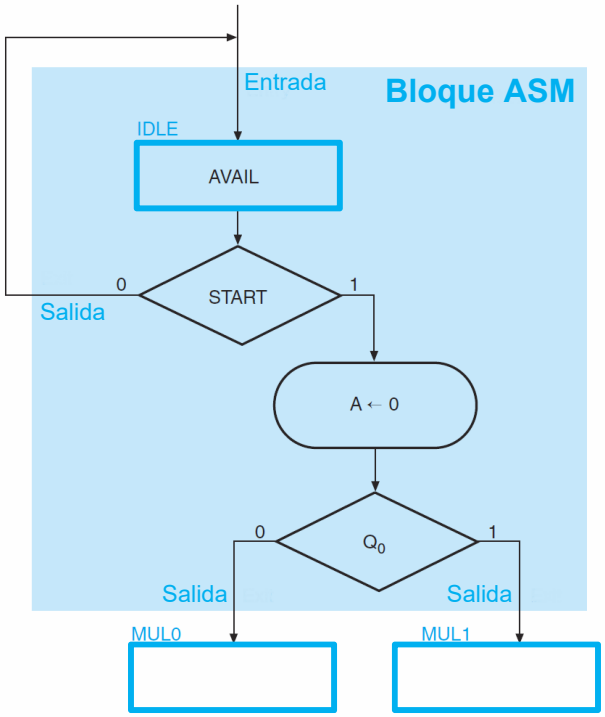
Salidas: AVAIL y CLRA (Reset síncrono de A).



DIAGRAMAS ASM
TEMPORIZACIÓN DE SEÑALES

Siguiendo con nuestro ejemplo:





ALGORITMO PARA LA MULTIPLICACIÓN

Como ejemplo, desarrollaremos un circuito para la multiplicación de números en binario natural sin signo.

Algoritmo:

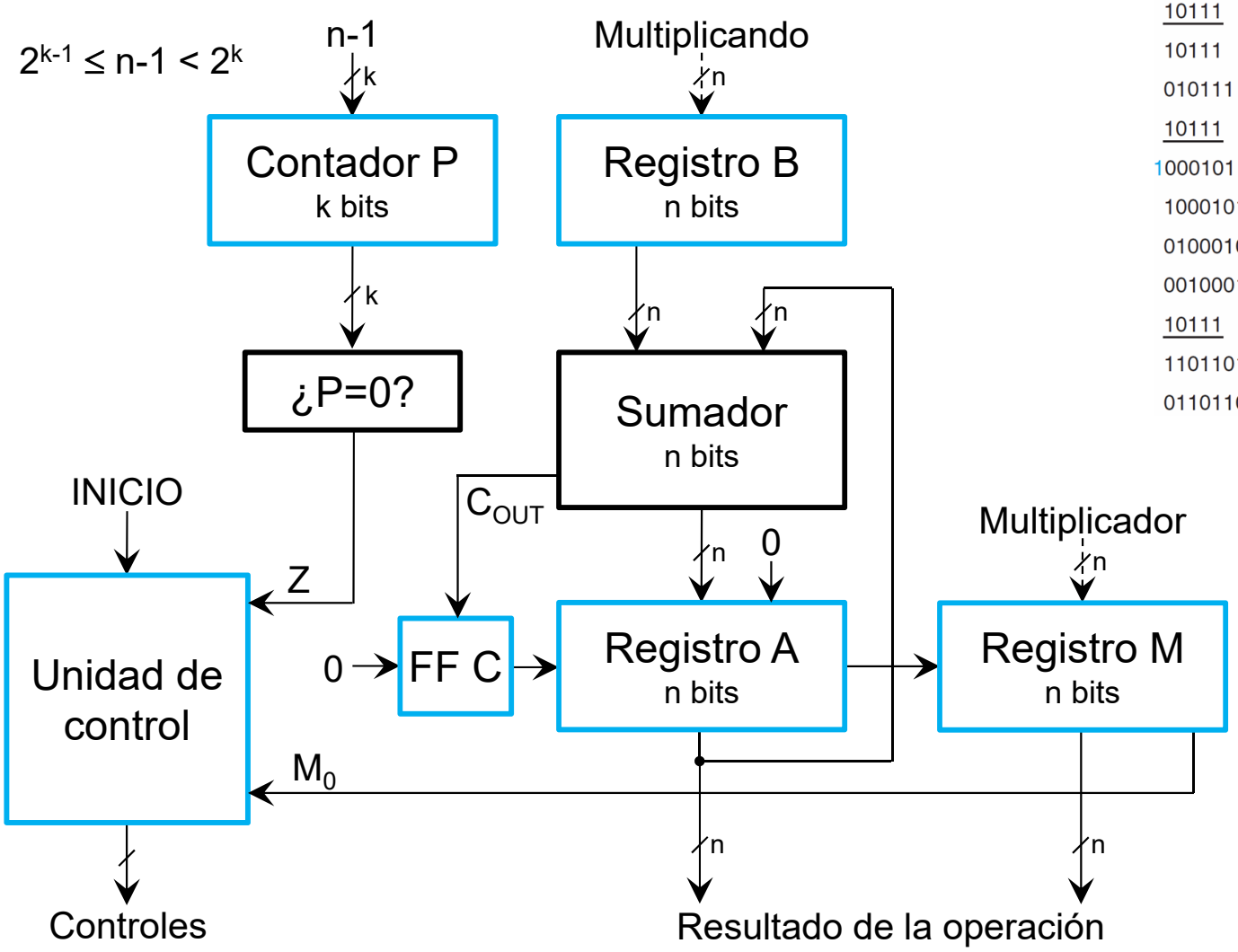
23	10111	Multiplicando
× 19	× 10011	Multiplicador
207	10111	Productos parciales
23	10111	
437	00000	
	00000	
	10111	
	110110101	Producto final

10111	Multiplicando
× 10011	Multiplicador
00000	Producto parcial inicial
10111	Suma el multiplicando pues el bit del multiplicador es 1
10111	Producto parcial después de sumar y antes de desplazar
010111	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
1000101	Producto parcial después de sumar y antes de desplazar
1000101	Producto parcial después de desplazar
01000101	Producto parcial después de desplazar
001000101	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
110110101	Producto parcial después de sumar y antes de desplazar
0110110101	Producto total después del desplazamiento final

ARQUITECTURA DE LA UNIDAD DE PROCESO

Se eligen los componentes que se van a emplear, a qué órdenes van a ser capaces de responder y cómo van a estar interconectados entre sí.

$2^{k-1} \leq n-1 < 2^k$



10111	Multiplicando
×10011	Multiplicador
00000	Producto parcial inicial
10111	Suma el multiplicando pues el bit del multiplicador es 1
10111	Producto parcial después de sumar y antes de desplazar
010111	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
1000101	Producto parcial después de sumar y antes de desplazar
1000101	Producto parcial después de desplazar
01000101	Producto parcial después de desplazar
001000101	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
110110101	Producto parcial después de sumar y antes de desplazar
0110110101	Producto total después del desplazamiento final

Operaciones:

Registro B: B ← Mtndo (asín)

Registro M: M ← Mtdor (asín)

Desplazamiento a la dcha de C, A y M: C‖A‖M ← sr C‖A‖M

Flip-flop C $\begin{cases} C \leftarrow C_{OUT} \\ C \leftarrow 0 \end{cases}$

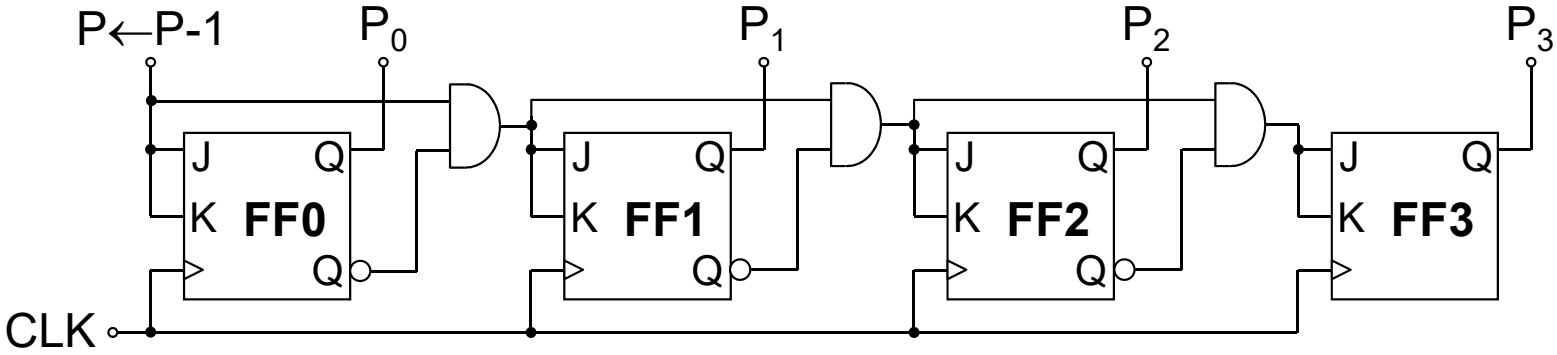
Registro A $\begin{cases} A \leftarrow A+B \\ A \leftarrow 0 \end{cases}$

Contador P $\begin{cases} P \leftarrow n-1 \\ P \leftarrow P-1 \end{cases}$

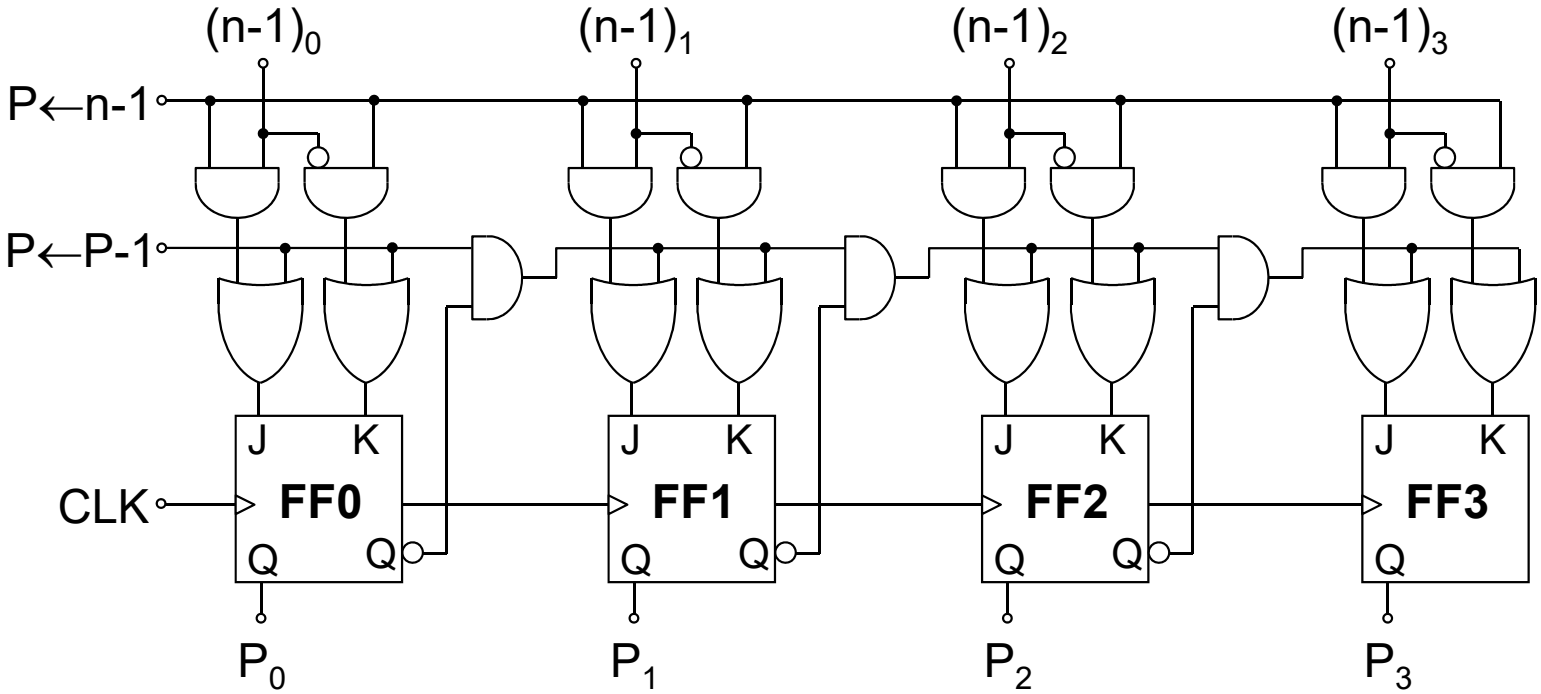
ARQUITECTURA DE LA UNIDAD DE PROCESO

CONTADOR P

Est.	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1



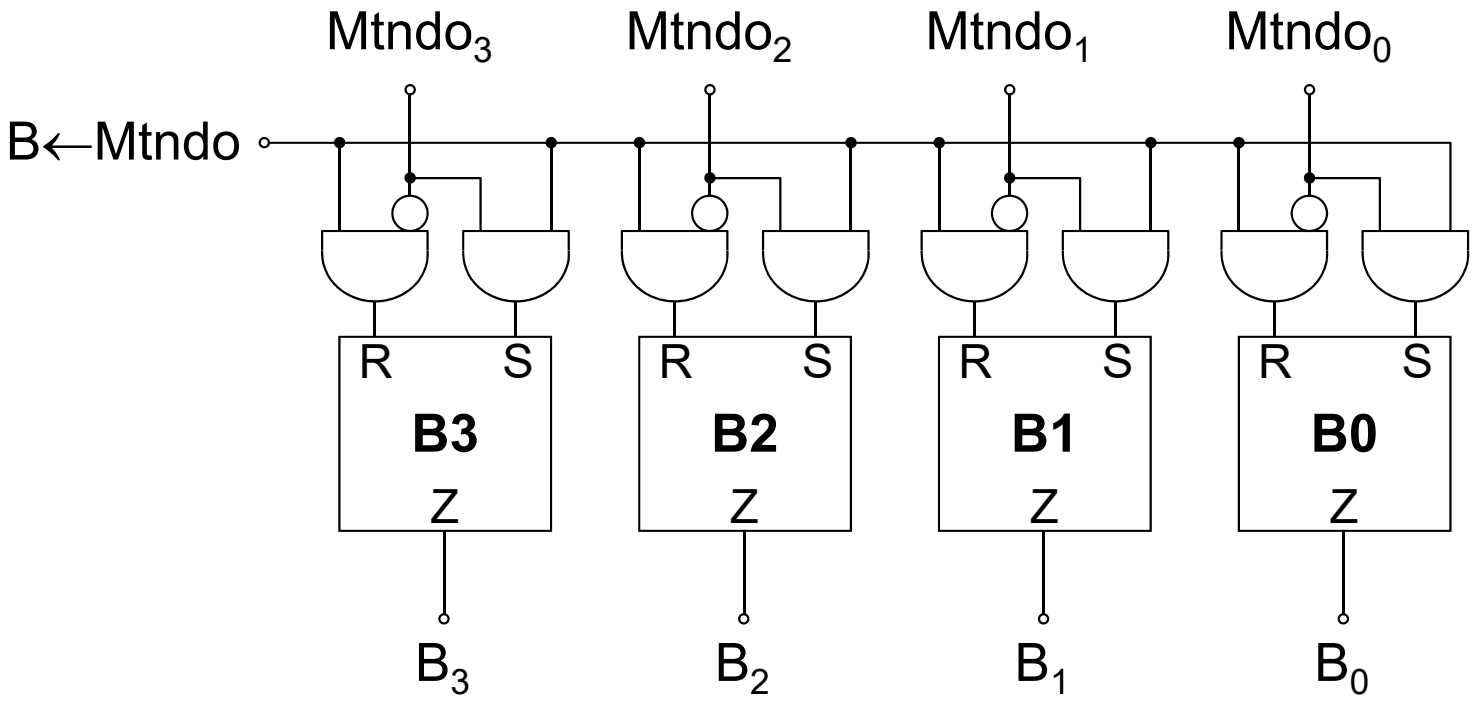
Añadimos una operación de carga en paralelo síncrona para el número n-1:



ARQUITECTURA DE LA UNIDAD DE PROCESO

REGISTRO B

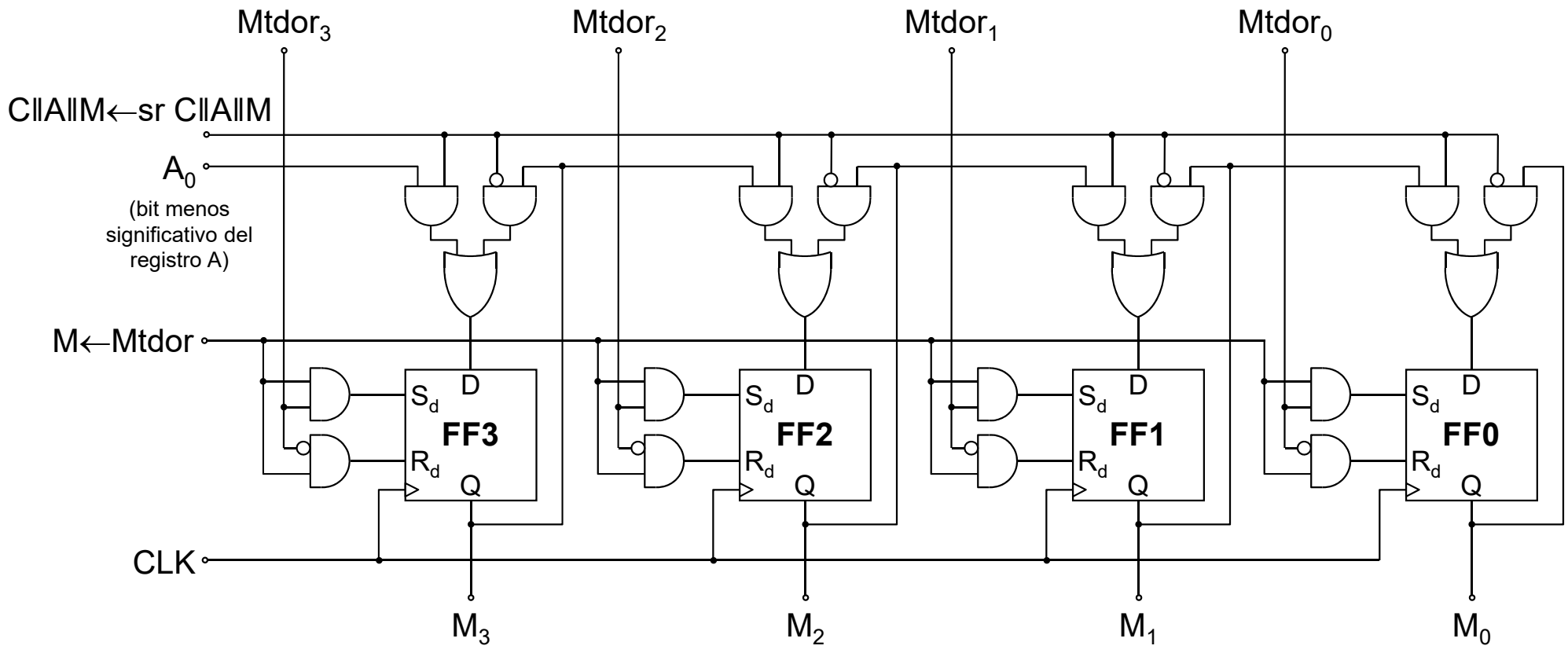
El registro B sólo se utiliza para almacenar el multiplicando de forma asíncrona, por lo que basta con implementarlo a partir de biestables RS:



ARQUITECTURA DE LA UNIDAD DE PROCESO

REGISTRO M

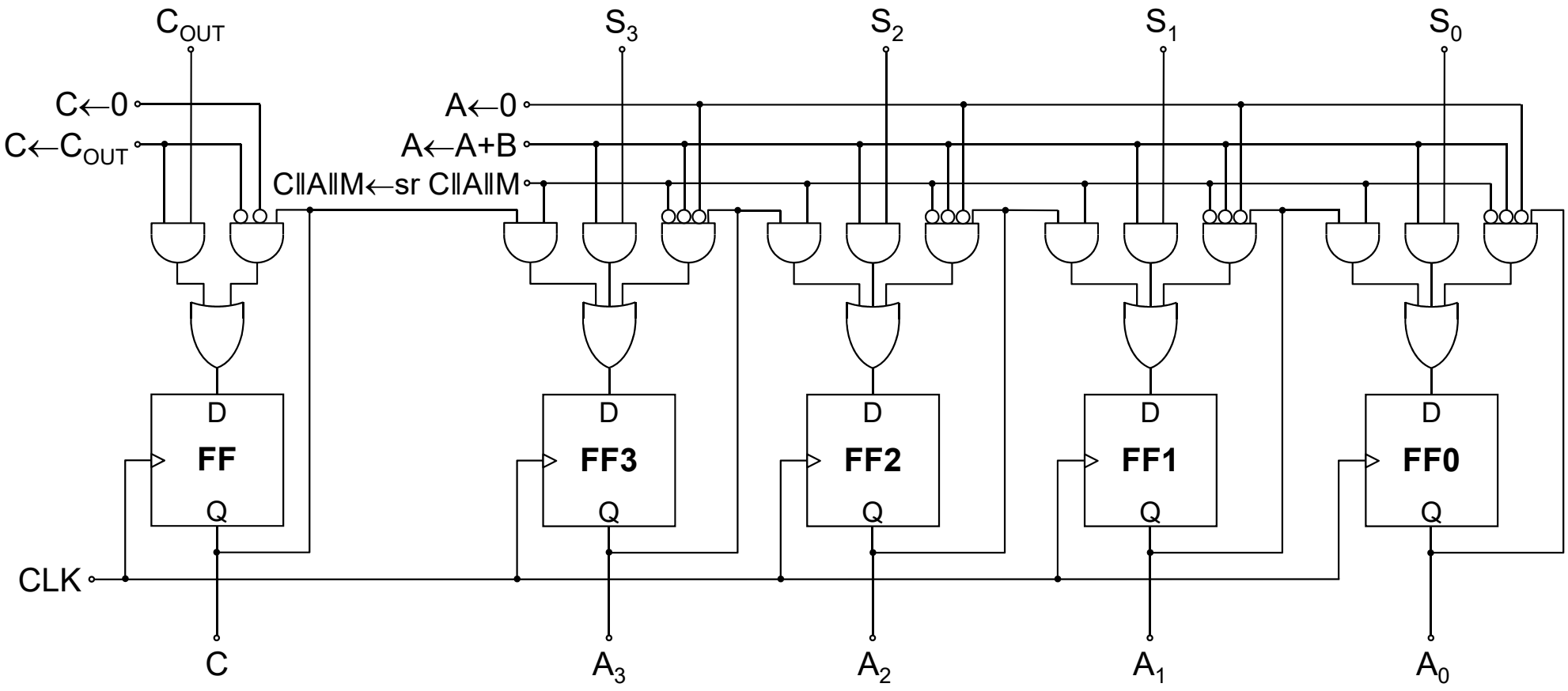
El registro M, además de permitir la carga asíncrona del multiplicador, debe implementar una operación de desplazamiento a la derecha:



ARQUITECTURA DE LA UNIDAD DE PROCESO

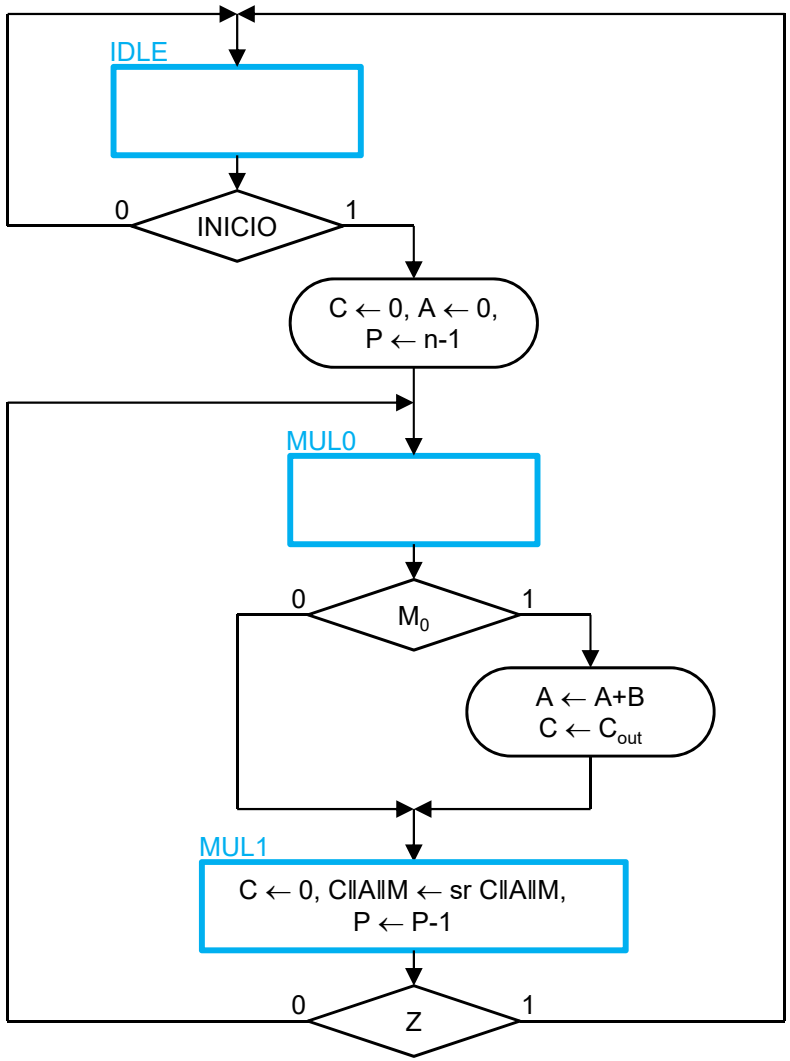
FLIP-FLOP C Y REGISTRO A

El flip-flop C y el registro A deben ser capaces de hacer una carga en paralelo síncrona de las salidas del sumador. Además, implementan una operación de desplazamiento a la derecha y otra de puesta a cero síncrona:

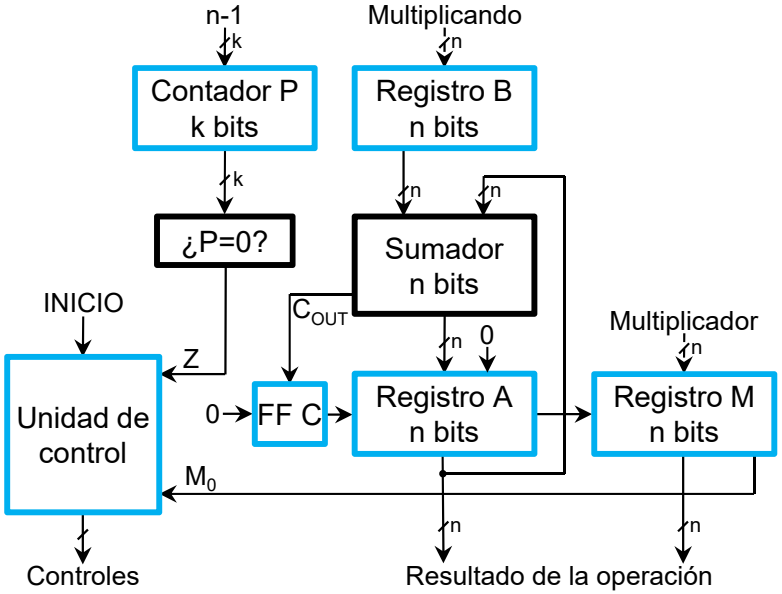


DISEÑO DEL CONTROLADOR

Una vez decidida la arquitectura de la unidad de proceso, se plantea el diagrama ASM del controlador:

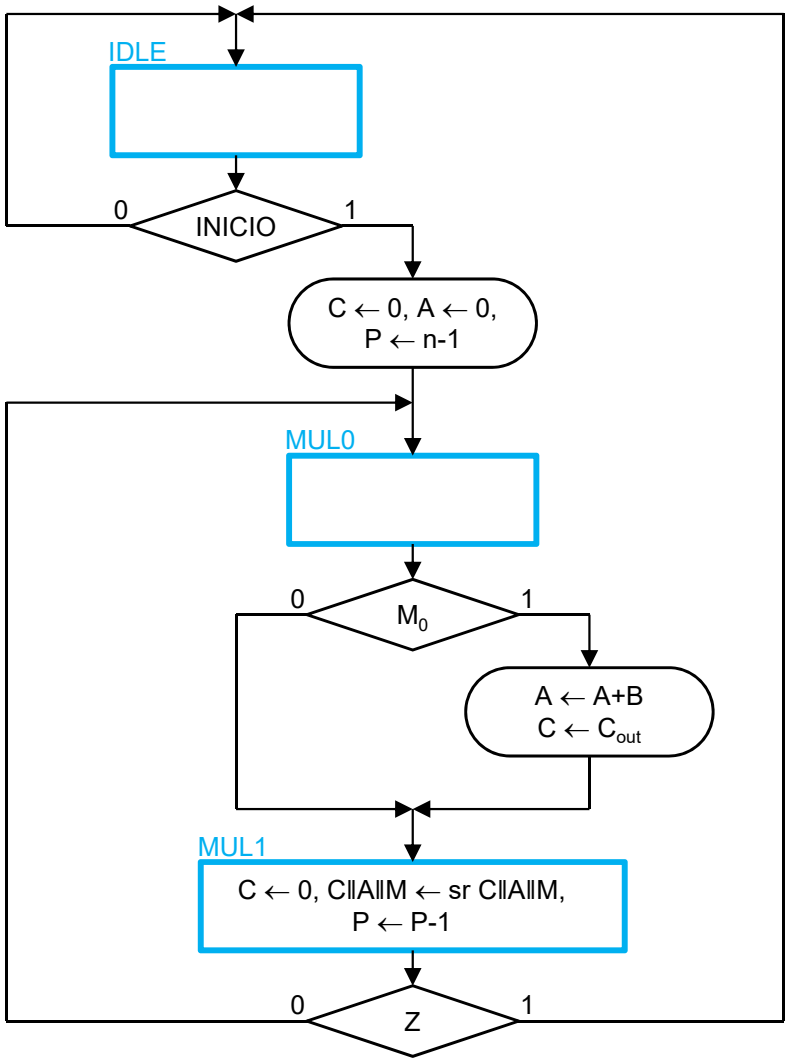


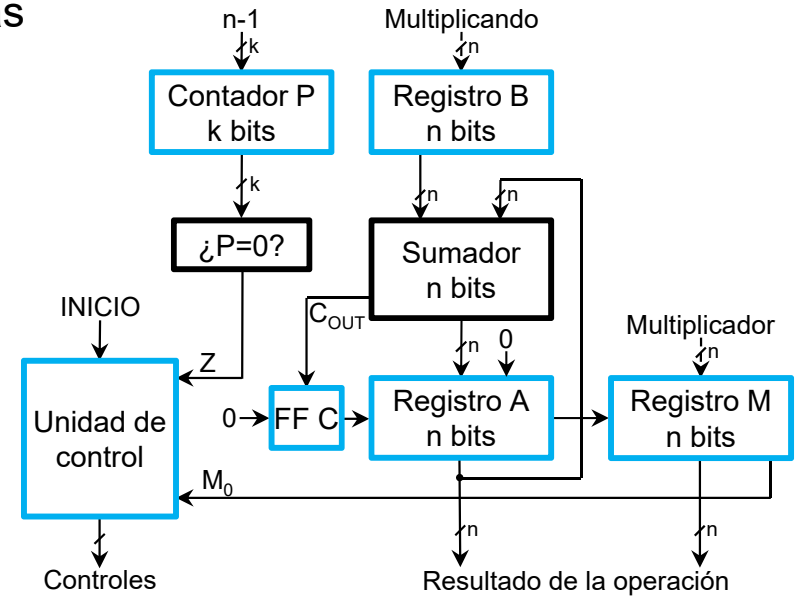
10111	Multiplicando
× 10011	Multiplicador
00000	Producto parcial inicial
10111	Suma el multiplicando pues el bit del multiplicador es 1
10111	Producto parcial después de sumar y antes de desplazar
010111	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
1000101	Producto parcial después de sumar y antes de desplazar
1000101	Producto parcial después de desplazar
01000101	Producto parcial después de desplazar
001000101	Producto parcial después de desplazar
10111	Suma el multiplicando pues el bit del multiplicador es 1
110110101	Producto parcial después de sumar y antes de desplazar
0110110101	Producto total después del desplazamiento final



SEÑALES DE CONTROL

Teniendo en cuenta el diagrama ASM, definiremos las señales de control para nuestra unidad de proceso:

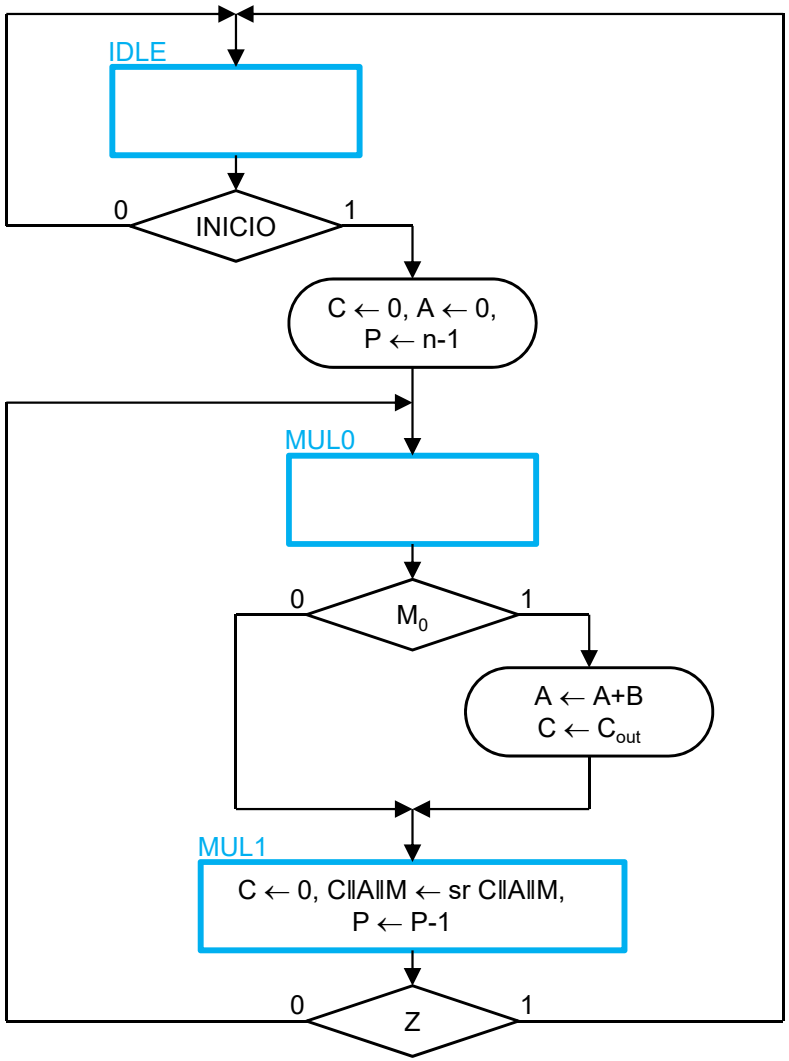


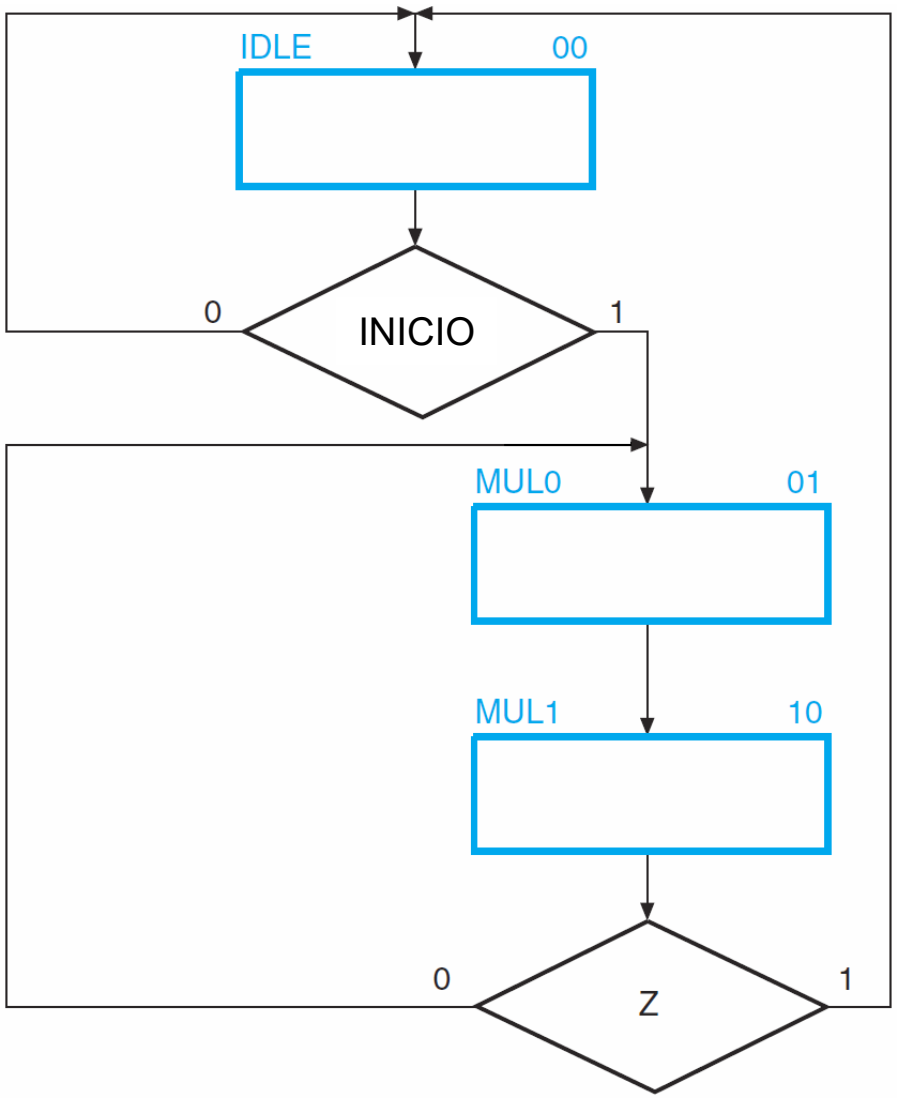


Módulo	Microperaciones	Controles	Expresiones
Registro A	A←0 A←A+B C‖A‖M←sr C‖A‖M	Initialize Load Shitf_dec	IDLE·INICIO MUL0·M_0 MUL1
Flip-flop C	C←0 C←C_{OUT}	Clear_C Load	IDLE·INICIO+MUL1 MUL0·M_0
Registro M	C‖A‖M←sr C‖A‖M	Shift_dec	MUL1
Contador P	P←n-1 P←P-1	Initialize Shift_dec	IDLE·INICIO MUL1

SECUENCIACIÓN DE LA UNIDAD DE CONTROL

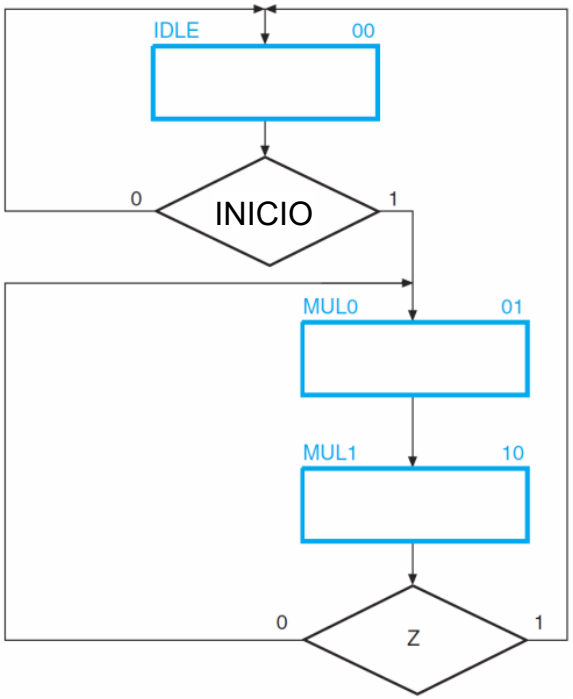
Eliminando la información relativa a las microoperaciones en el diagrama ASM, nos quedaremos con la correspondiente a la secuenciación de estados:



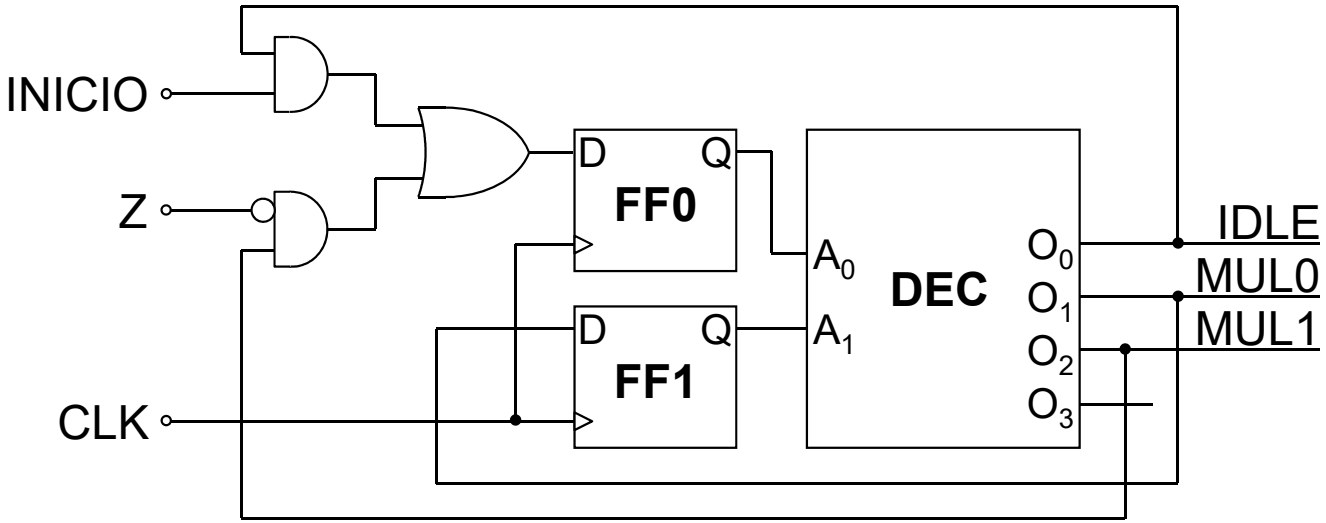


REGISTRO DE SECUENCIA Y DECODIFICADOR

Al tener tres estados, necesitamos 2 flip-flops y un decodificador de dos bits de dirección:



Estado actual		Entradas		Est. Sig.		Salidas decodificador		
Nombre	Q_1^n Q_0^n	INICIO	Z	Q_1^{n+1}	Q_0^{n+1}	IDLE	MUL0	MUL1
IDLE	0 0	0	*	0	0	1	0	0
	0 0	1	*	0	1	1	0	0
MUL0	0 1	*	*	1	0	0	1	0
MUL1	1 0	*	0	0	1	0	0	1
	1 0	*	1	0	0	0	0	1
——	1 1	*	*	*	*	*	*	*

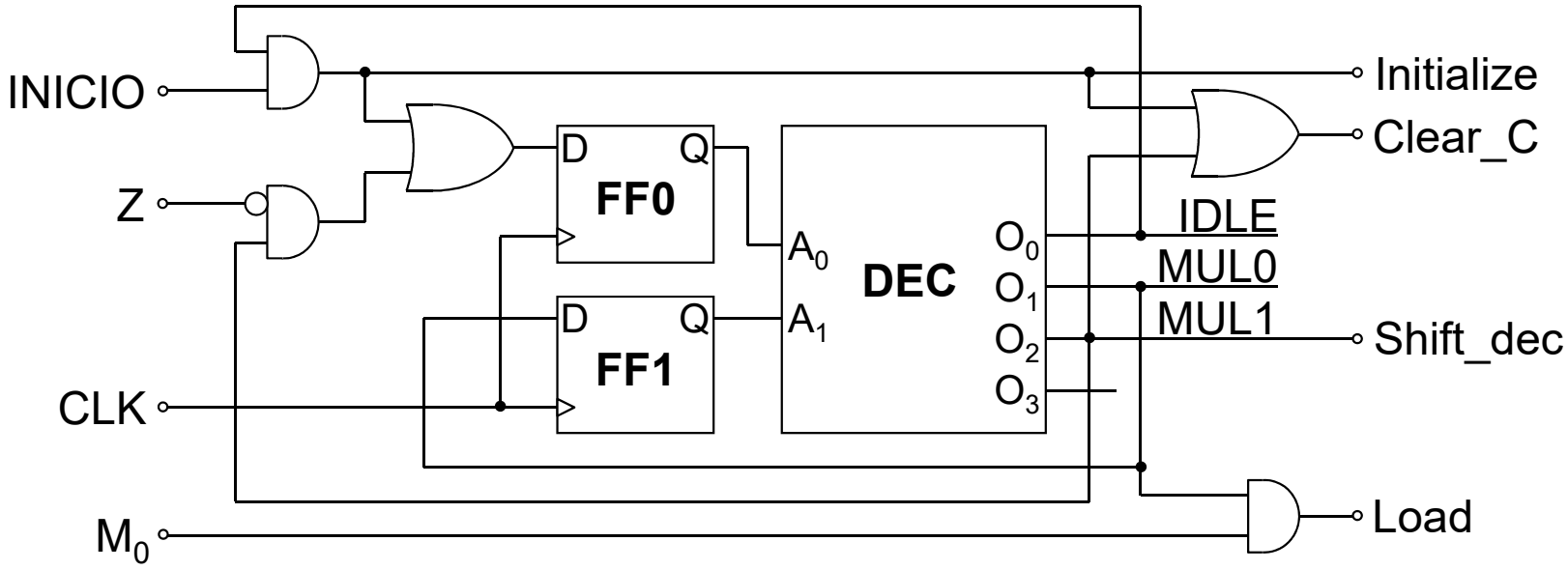


Los flip-flops se inicializan a 0 a través de sus entradas de reset directas para que el sistema empiece a funcionar en el estado IDLE.

REGISTRO DE SECUENCIA Y DECODIFICADOR

Teniendo en cuenta la tabla que definía los controles para la unidad de proceso, añadimos la circuitería para generar las señales de control:

Módulo	Microperaciones	Controles	Expresiones
Registro A	A←0 A←A+B C‖A‖M←sr C‖A‖M	Initialize Load Shitf_dec	IDLE·INICIO MUL0·M_0 MUL1
Flip-flop C	C←0 C←C_{OUT}	Clear_C Load	IDLE·INICIO+MUL1 MUL0·M_0
Registro M	C‖A‖M←sr C‖A‖M	Shift_dec	MUL1
Contador P	P←n-1 P←P-1	Initialize Shift_dec	IDLE·INICIO MUL1

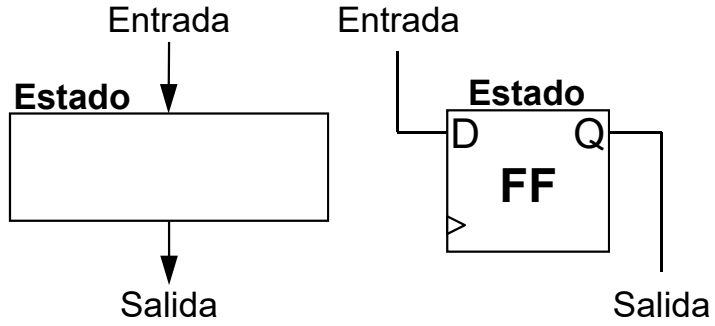


DISEÑO CON UN REGISTRO DE DESPLAZAMIENTO

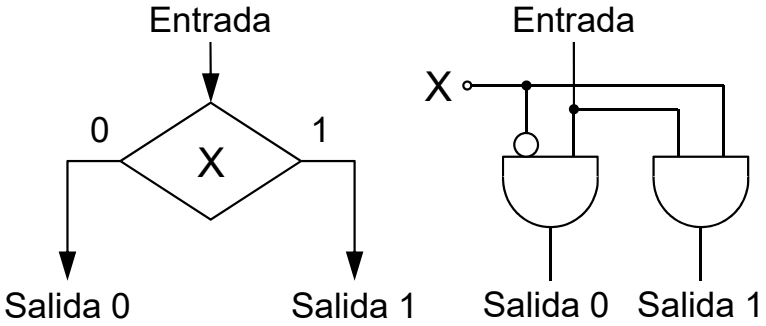
En vez de codificar, se utiliza un flip-flop por cada estado. El sistema se encontrará en un estado determinado cuando el flip-flop correspondiente esté en 1 (los demás deberán estar en 0). El registro describirá el comportamiento del controlador si el 1 se desplaza por los flip-flops del mismo modo que tiene lugar la evolución temporal indicada en el diagrama ASM.

Aunque se necesitan muchos más flip-flops, el diseño se simplifica debido a la equivalencia directa entre los elementos del diagrama ASM y el circuito del registro de desplazamiento:

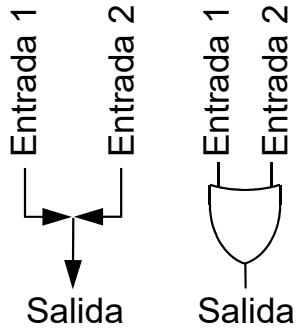
Caja de estado:



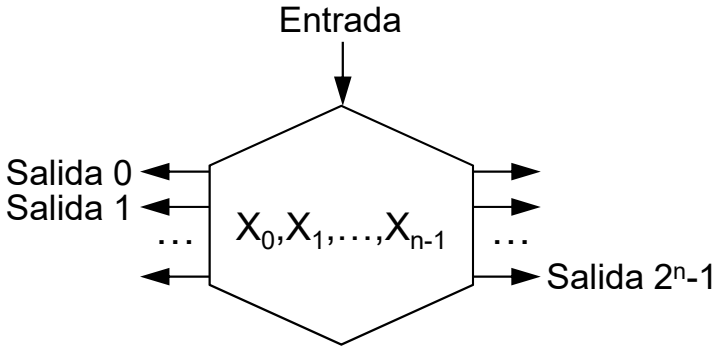
Caja de decisión escalar:

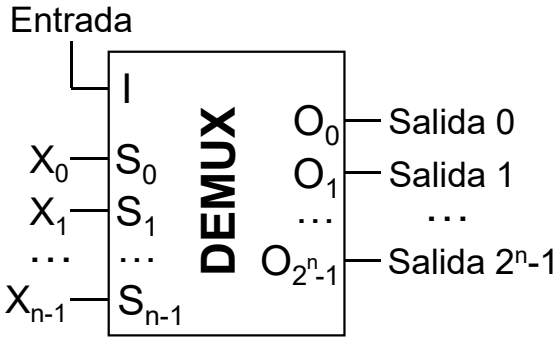


Unión:

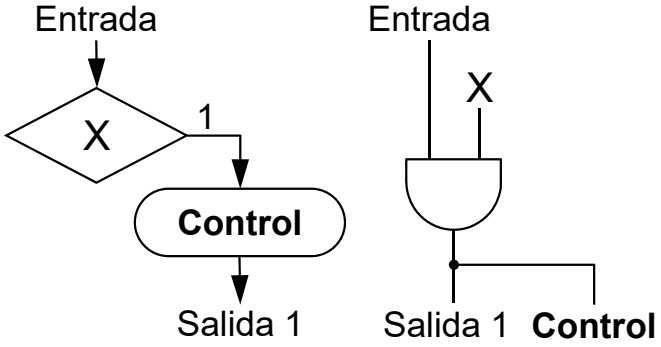


Caja de decisión vectorial:



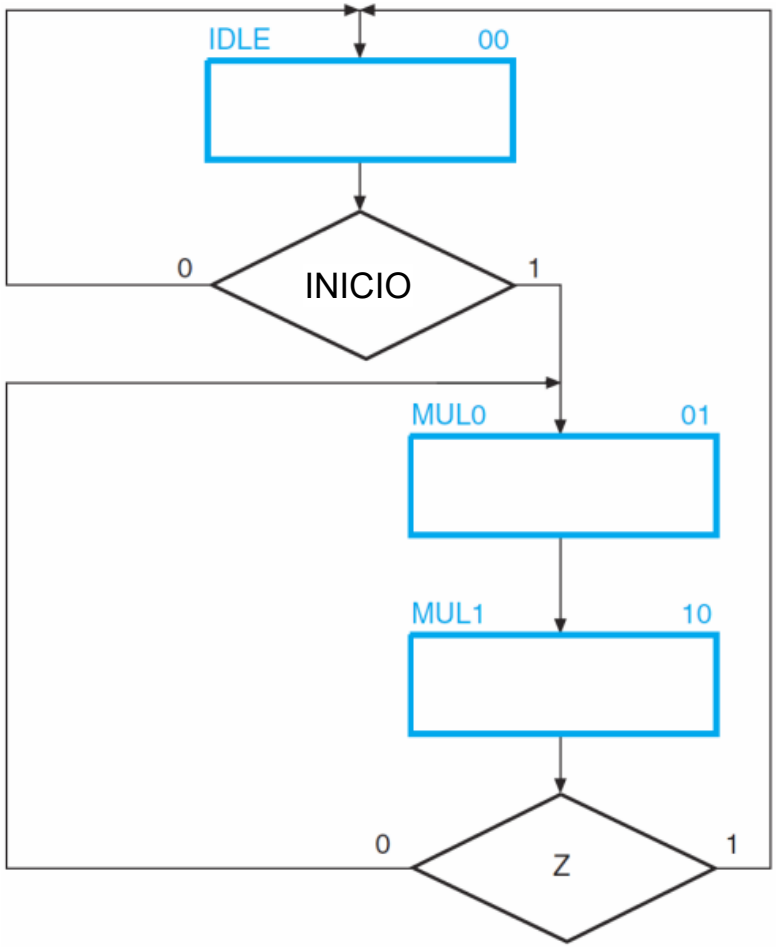


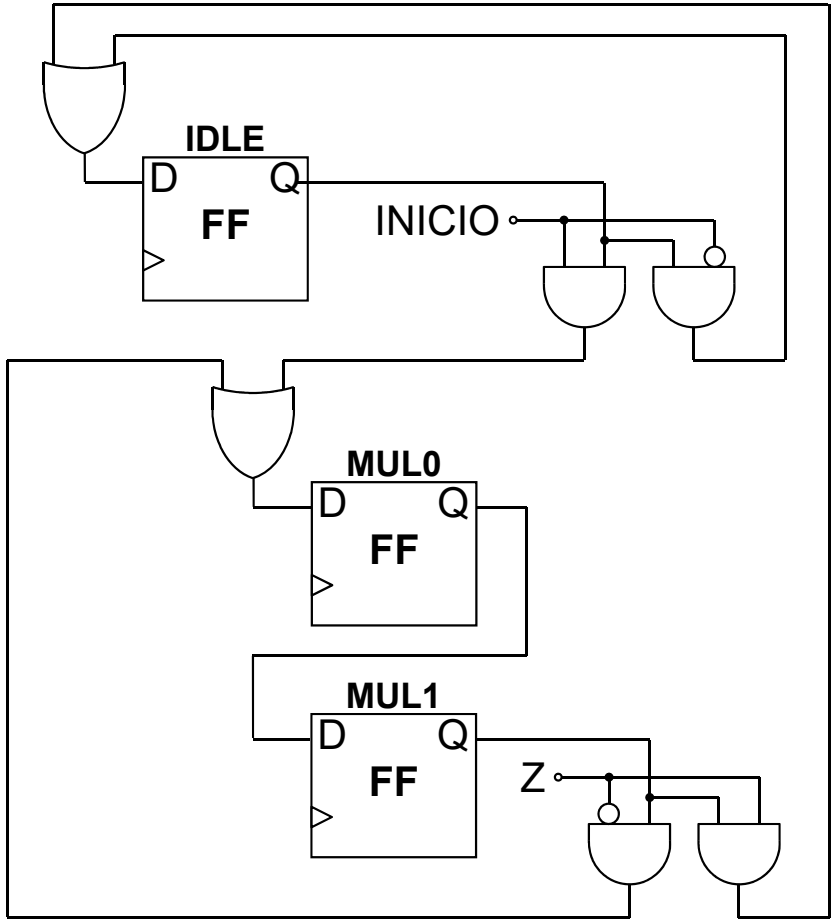
Caja de salida condicional:



DISEÑO CON UN REGISTRO DE DESPLAZAMIENTO

Consideramos primeramente la parte secuencial del controlador:

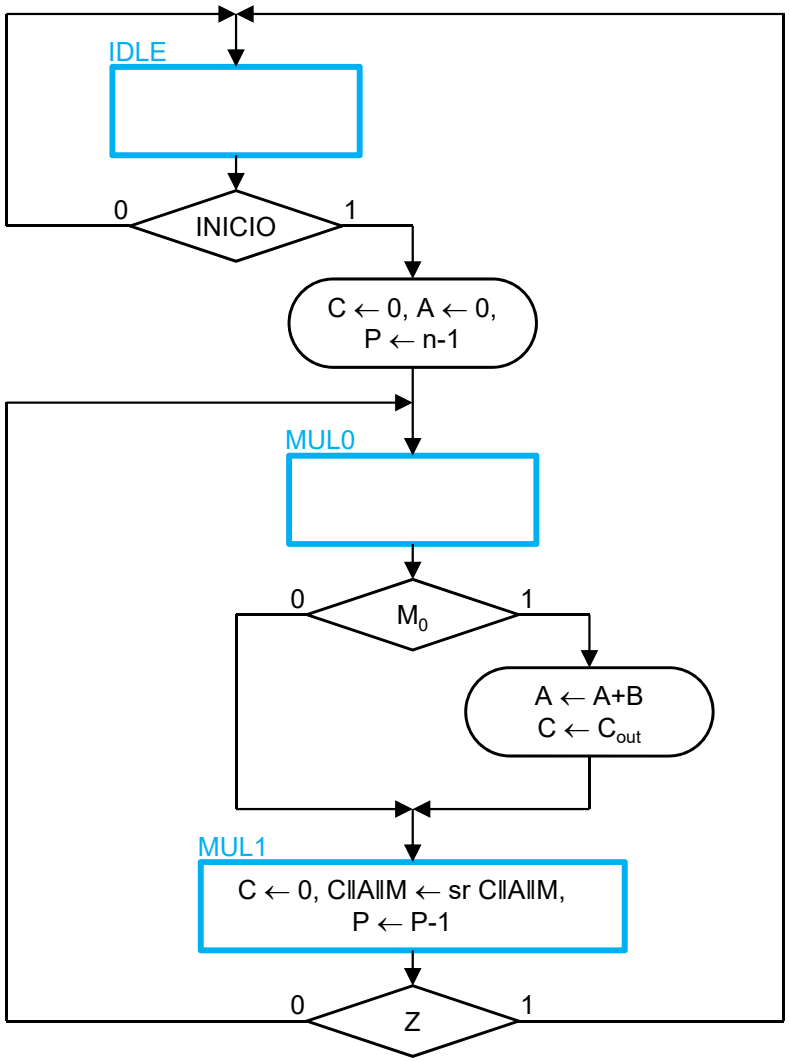




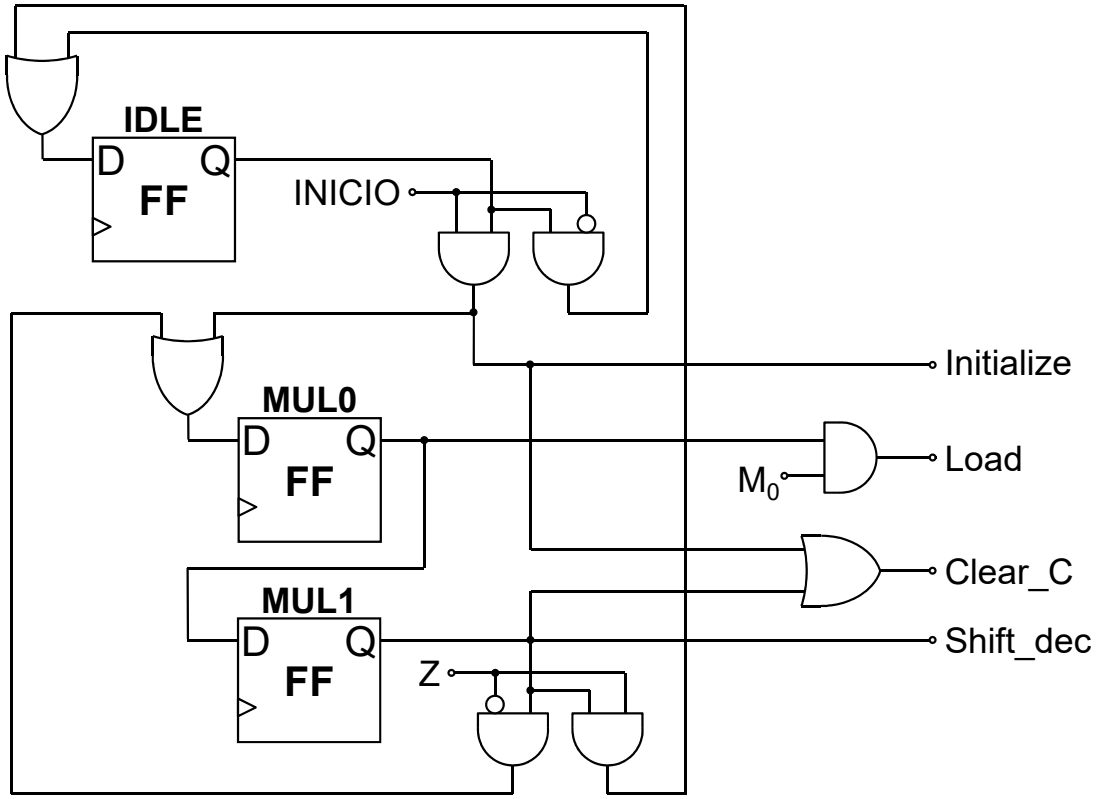
Para que el circuito funcione correctamente, el flip-flop IDLE hay que inicializarlo en 1, y el resto en 0.

DISEÑO CON UN REGISTRO DE DESPLAZAMIENTO

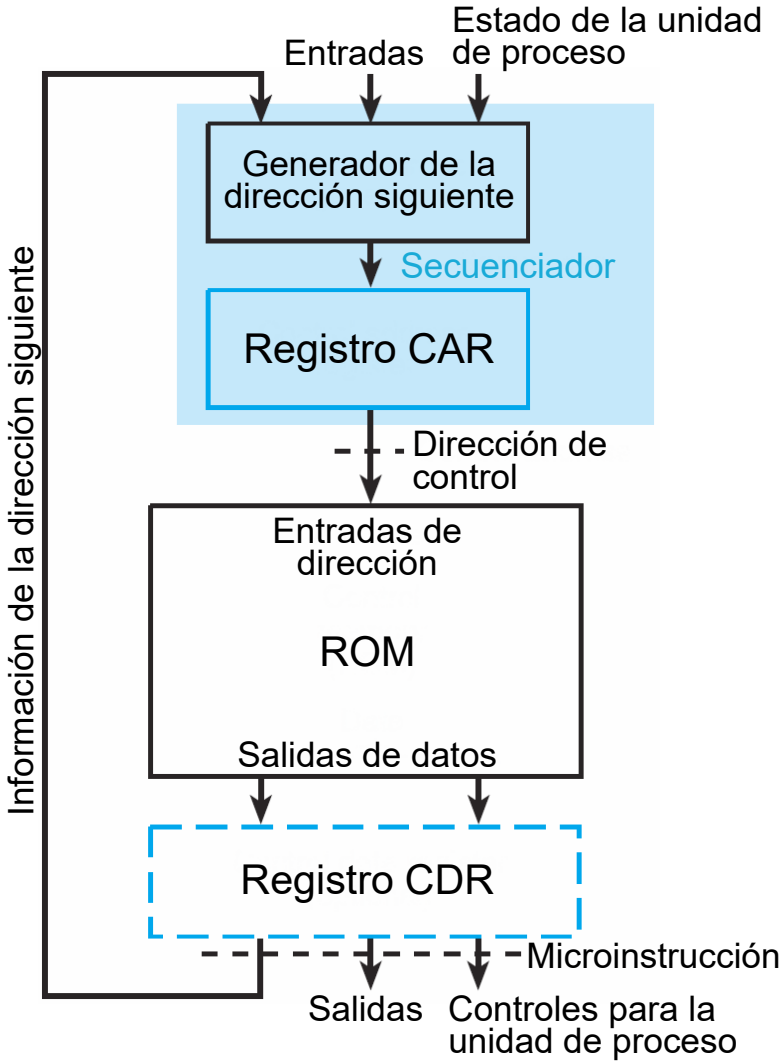
Para implementar las salidas de control, tendremos en cuenta el diagrama ASM completo:



Microperaciones	Controles	Expresiones
A←0, P←n-1	Initialize	IDLE·INICIO
A←A+B, C←C_{OUT}	Load	MUL0·M_0
C‖A‖M←sr C‖A‖M, P←P-1	Shitf_dec	MUL1
C←0	Clear_C	IDLE·INICIO+MUL1



Un *controlador microprogramado* es aquél que tiene sus valores de control almacenados como palabras en una memoria (normalmente una ROM). Cada palabra de control almacenada en la memoria contiene una *mircroinstrucción* que especifica una o varias microoperaciones. Una secuencia de microinstrucciones constituye un *microprograma*.



El *registro de control de direcciones* CAR (Control Address Register) almacena la dirección de la ROM en que se encuentra la microinstrucción a ejecutar.

El *registro de control de datos* CDR (Control Data Register), opcional, almacena la microinstrucción en curso.

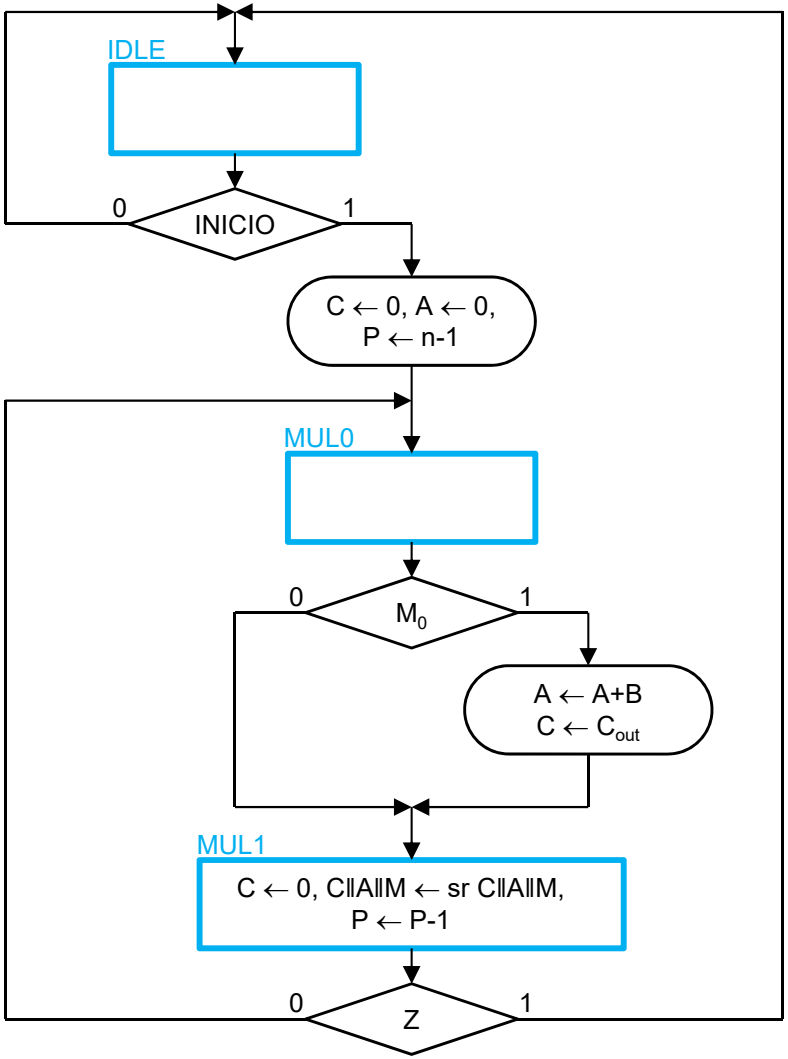
La microinstrucción contiene los valores de salida del controlador, así como la información necesaria para determinar la microinstrucción siguiente.

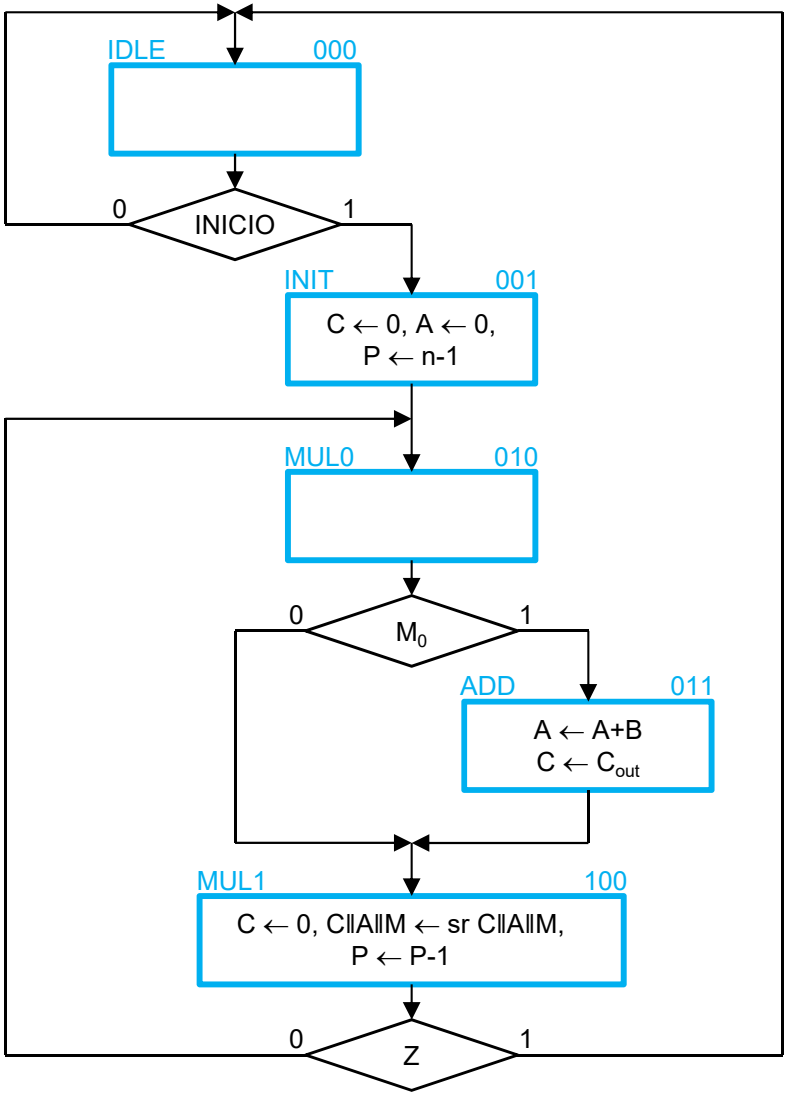
Esta información, junto con las entradas al controlador, llegan a un combinacional que genera la dirección de la siguiente microinstrucción a ejecutar, la cual pasa al CAR con el flanco activo de la señal de reloj. Dicho combinacional, junto con el CAR, constituyen el *secuenciador*.

Debido a que el estado de la unidad de proceso no actúa directamente sobre la microinstrucción a ejecutar, este esquema sólo permite circuitos de Moore ⇒ En el diagrama ASM no puede haber cajas de salida condicional.

CIRCUITO MULTIPLICADOR MICROPROGRAMADO

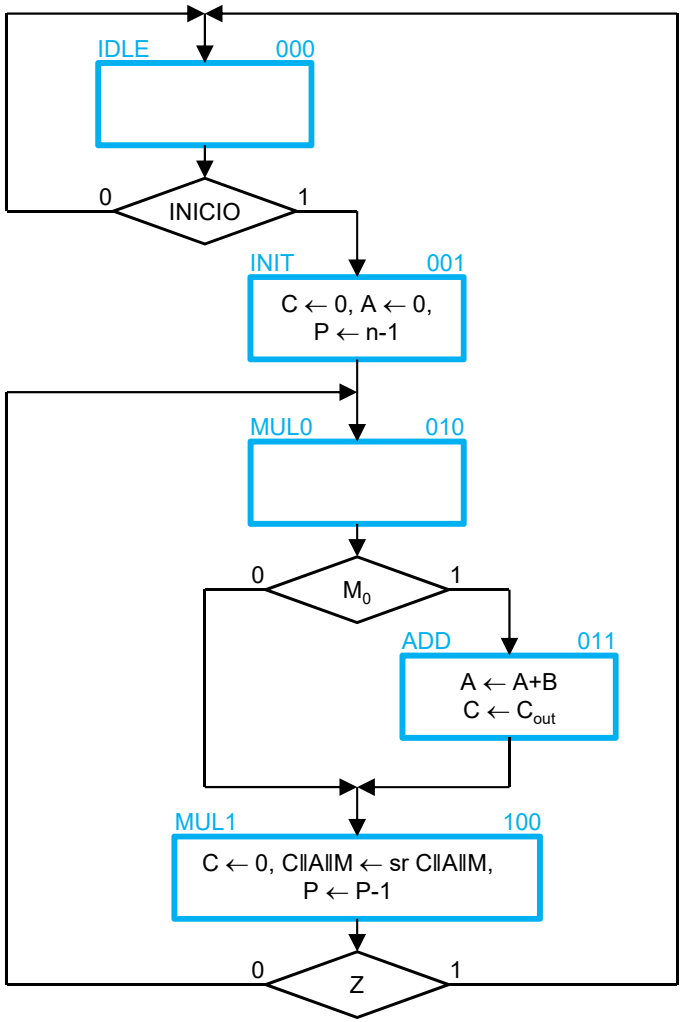
Primero transformamos el diagrama ASM original a uno tipo Moore:



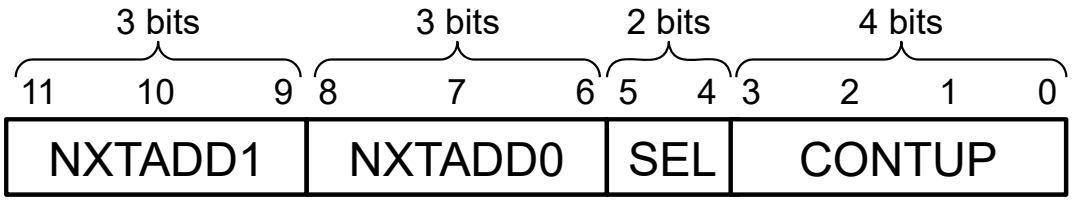


CIRCUITO MULTIPLICADOR MICROPROGRAMADO

PALABRA DE CONTROL



La palabra de control (microinstrucción, μI) constará de los siguientes campos:



CONTUP: Las salidas de control para la unidad de proceso. Son 4 bits, ya que tenemos 4 señales de control para la unidad de proceso: Initialize, Load, Shift_dec y Clear_C.

SEL: Código para la selección de las variables de control de la secuenciación. Son dos bits, puesto que tenemos 3 variables respecto a las que se hacen bifurcaciones en la unidad de control: INICIO, M_0 y Z.

NXTADD0 y NXTADD1: Direcciones de las siguientes microinstrucciones a ejecutar, la primera si la variable de control de la secuenciación es 0 y la segunda si es 1. Hay tantas microinstrucciones como estados distintos tiene la unidad de control. Por tanto se necesitan 3 bits para direccionar los cinco estados: IDLE, INIT, MUL0, ADD y MUL1.

Por tanto, el CAR será un registro de 3 bits, y la ROM tendrá una capacidad mínima de 2^3x12 bits.

CIRCUITO MULTIPLICADOR MICROPROGRAMADO

PALABRA DE CONTROL

11 10 9	8 7 6	5 4	3 2 1 0
NXTADD1	NXTADD0	SEL	CONTUP

Señales de control para la unidad de proceso:

Controles	Microperaciones	Estado UC	Bit μI	Símbolo
Initialize	A←0, P←n-1	INIT	0	IT
Load	A←A+B, C←C_{OUT}	ADD	1	LD
Shift_dec	C‖A‖M←sr C‖A‖M, P←P-1	MUL1	2	SD
Clear_C	C←0	INIT, MUL1	3	CC

Códigos del campo SEL:

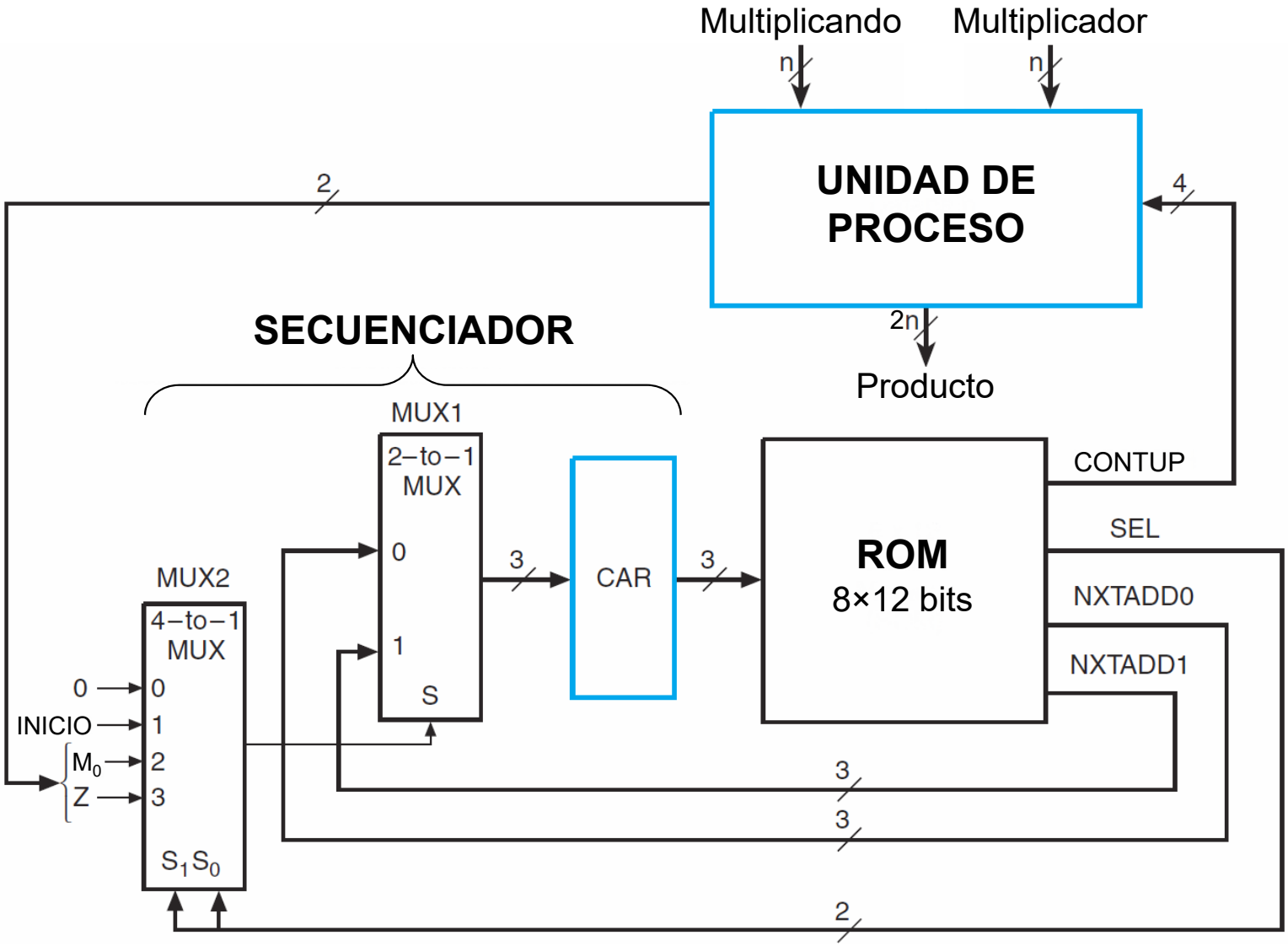
Símbolo	Código	Microoperaciones
NXT	0 0	CAR←NXTADD0
DIN	0 1	$\overline{\text{INICIO}}$: CAR ←NXTADD0 INICIO: CAR ←NXTADD1
DM	1 0	$\overline{M_0}$: CAR ←NXTADD0 M_0: CAR ←NXTADD1
DZ	1 1	$\overline{Z}$: CAR ←NXTADD0 Z: CAR ←NXTADD1

Códigos de dirección de los estados del controlador:

Estado	Dirección
IDLE	0 0 0
INIT	0 0 1
MUL0	0 1 0
ADD	0 1 1
MUL1	1 0 0

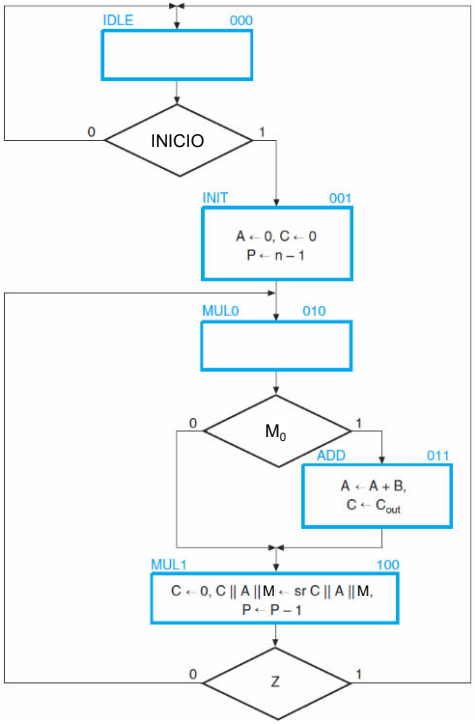
CIRCUITO MULTIPLICADOR MICROPROGRAMADO

ARQUITECTURA DE LA UNIDAD DE CONTROL



CIRCUITO MULTIPLICADOR MICROPROGRAMADO

MICROPROGRAMA



Microprograma descrito mediante microoperaciones de transferencia entre registros:

Dirección	Microoperaciones
IDLE	$\underline{\text{INICIO}}$: CAR←INIT $\overline{\text{INICIO}}$: CAR←IDLE
INIT	A←0, C←0, P←n-1, CAR←MUL0
MUL0	$\underline{M_0}$: CAR←ADD $\overline{M_0}$: CAR←MUL1
ADD	A←A+B, C←C_{OUT}, CAR←MUL1
MUL1	C←0, C‖A‖M←sr C‖A‖M, P←P-1 $\underline{Z}$: CAR←IDLE $\overline{Z}$: CAR←MUL0

Microprograma simbólico:

Dirección	NXADD1	NXADD0	SEL	CONTUP
IDLE	INIT	IDLE	DIN	—
INIT	—	MUL0	NXT	IT, CC
MUL0	ADD	MUL1	DM	—
ADD	—	MUL1	NXT	LD
MUL1	IDLE	MUL0	DZ	CC, SD

Microprograma en binario:

Dirección	NXADD1	NXADD0	SEL	CONTUP
0 0 0	0 0 1	0 0 0	0 1	0 0 0 0
0 0 1	* * *	0 1 0	0 0	1 0 0 1
0 1 0	0 1 1	1 0 0	1 0	0 0 0 0
0 1 1	* * *	1 0 0	0 0	0 0 1 0
1 0 0	0 0 0	0 1 0	1 1	1 1 0 0